14 Configuration and secrets management

This chapter covers

- Configuring applications on Kubernetes
- Using ConfigMaps and Secrets in Kubernetes
- Managing deployments and configuration with Kustomize

Releasing applications to production involves two important aspects: an executable artifact and its configuration. The executable artifact could be a JAR file or a container image. The previous chapters covered several principles, patterns, and tools for building applications that are loosely coupled, resilient, scalable, secure, and observable. You saw how to package applications as executable JAR artifacts or container images. I also guided you through the implementation of the commit stage of a deployment pipeline, which ultimately produces a release candidate.

The other aspect of being ready for production is configuration. Chapter 4 introduced the importance of externalized configuration for cloud native applications and covered several techniques for configuring Spring Boot applications. This chapter will continue that discussion in preparation for deploying an entire cloud native system to a Kubernetes production environment.

First I'll describe a few options for configuring Spring Boot applications on Kubernetes and describe what's missing for using Spring Cloud Config in production. Then you'll learn how to use ConfigMaps and Secrets, a native mechanism for handling configuration on Kubernetes. As part of the discussion, you'll get to know Spring Cloud Kubernetes and its primary use cases. Finally, I'll expand on configuration and secrets management for production workloads on Kubernetes, and you'll learn how to implement that using Kustomize.

NOTE The source code for the examples in this chapter is available in the Chapter14/14-begin and Chapter14/14-end folders, containing the initial and final states of the project (**https://github.com/ThomasVitale/cloud-native-spring-in-action**).

14.1 Configuring applications on Kubernetes

According to the 15-Factor methodology, configuration is anything that changes between deployment environments. We started working with configuration in chapter 4 and since then have used different configuration strategies:

- *Property files packaged with the application*—These can act as specifications of what configuration data the application supports, and they are useful for defining sensible default values, mainly oriented to the development environment.
- *Environment variables*—These are supported by any operating system, so they are great for portability. They're useful for defining configuration data depending on the infrastructure or platform where the application is deployed, such as active profiles, hostnames, service names, and port numbers. We used them in Docker and Kubernetes.
- *Configuration service*—This provides configuration data persistence, auditing, and accountability. It's useful for defining configuration data specific to the application, such as feature flags, thread pools, connection pools, timeouts, and URLs for third-party services. We adopted this strategy with Spring Cloud Config.

Those three strategies are generic enough that we can use them to configure applications for any cloud environment and service model (CaaS, PaaS, FaaS). When it comes to Kubernetes, there's an additional configuration strategy that is provided natively by the platform: ConfigMaps and Secrets.

These are a very convenient way to define configuration data that depends on the infrastructure and platform where the application is deployed: service names (defined by Kubernetes Service objects), credentials and certificates for accessing other services running on the platform, graceful shutdown, logging, and monitoring. You could use ConfigMaps and Secrets to complement or completely replace what a configuration service does. Which you choose depends on the context. In any case, Spring Boot provides native support for all those options.

For the Polar Bookshop system, we'll use ConfigMaps and Secrets instead of the Config Service to configure applications in Kubernetes environments. Still, all the work we've done so far on Config Service would make including it in the overall deployment of Polar Bookshop on Kubernetes straightforward. In this section, I'll share some final considerations for making Config Service production-ready, in case you'd like to expand on the examples and include it in the final deployment in production.

4.1.1 Securing the configuration server with Spring Security

In previous chapters, we spent quite some time ensuring a high-security level for the Spring Boot applications in Polar Bookshop. However, Config Service was not one of them, and it's still unprotected. Even if it's a config server, it's still a Spring Boot application at its heart. As such, we can secure it using any of the strategies provided by Spring Security.

Config Service is accessed over HTTP by the other Spring Boot applications in the architecture. Before using it in production, we must ensure that only authenticated and authorized parties can retrieve configuration data. One option would be to use the OAuth2 Client credentials flow to secure the interactions between Config Service and applications based on an Access Token. It's an OAuth2 flow specific for protecting service-to-service interactions.

Assuming that applications will communicate over HTTPS, the *HTTP Basic* authentication strategy would be another viable option. When using this strategy, applications can be configured with the username and password via the properties exposed by Spring Cloud Config Client: `spring.cloud.config.username` and `spring.cloud.config.password`. For more information, refer to the official documentation for Spring Security (**https://spring.io/projects/spring-security**) and Spring Cloud Config (**https://spring.io/projects/spring-cloud-config**).

4.1.2 Refreshing configuration at runtime with Spring Cloud Bus

Imagine you have deployed your Spring Boot applications in a cloud environment like Kubernetes. During the startup phase, each application loaded its configuration from an external config server, but at some point you decide to make changes in the config repo. How can you make the applications aware of the configuration changes and have them reload it?

In chapter 4, you learned that you could trigger a configuration refresh operation by sending a `POST` request to the `/actuator/refresh` endpoint provided by Spring Boot Actuator. A request to that endpoint results in a `RefreshScopeRefreshedEvent` event inside the application context. All beans marked with `@ConfigurationProperties` or `@RefreshScope` listen to that event and get reloaded when it happens.

You tried the refresh mechanism on Catalog Service, and it worked fine, since it was just one application, and not even replicated. How about in production? Considering the dis-

tribution and scale of cloud native applications, sending an HTTP request to all the instances of each application might be a problem. Automation is a crucial part of any cloud native strategy, so we need a way to trigger a `RefreshScopeRefreshedEvent` event in all of them in one shot. There are a few viable solutions. Using Spring Cloud Bus is one of them.

Spring Cloud Bus (**https://spring.io/projects/spring-cloud-bus**) establishes a convenient communication channel for broadcasting events among all the application instances linked to it. It provides an implementation for AMQP brokers (like RabbitMQ) and Kafka, relying on the Spring Cloud Stream project you learned about in chapter 10.

Any configuration change consists of pushing a commit to the config repo. It would be convenient to set up some automation to make Config Service refresh the configuration when a new commit is pushed to the repository, completely removing the need for manual intervention. Spring Cloud Config provides a Monitor library that makes that possible. It exposes a `/monitor` endpoint that can trigger a configuration change event in Config Service, which then would send it over the Bus to all the listening applications. It also accepts arguments describing which files have been changed and supports receiving push notifications from the most common code repository providers like GitHub, GitLab, and Bitbucket. You can set up a webhook in those services to automatically send a `POST` request to Config Service after each new push to the config repo.

Spring Cloud Bus solves the problem of broadcasting a configuration change event to all connected applications. With Spring Cloud Config Monitor, we can further automate the refresh and make it happen after a configuration change is pushed to the repository backing the config server. This solution is illustrated in figure 14.1.

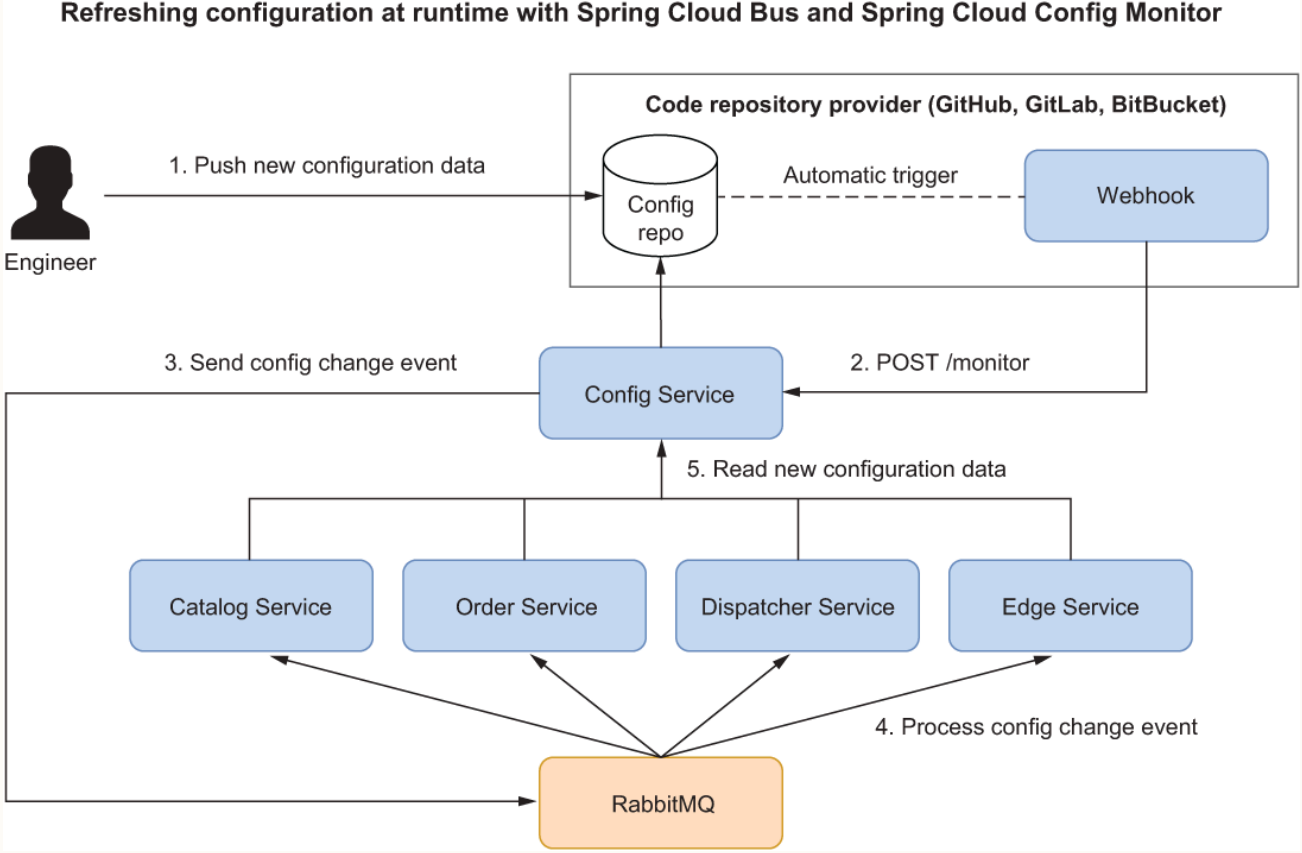


Figure 14.1 Broadcasting configuration changes through Spring Cloud Bus after the Config Service receives push notifications on every config repo change.

NOTE You can rely on Spring Cloud Bus to broadcast configuration changes even when you use other options like Consul (with Spring Cloud Consul), Azure Key Vault (Spring Cloud Azure), AWS Parameter Store or AWS Secrets Manager (Spring Cloud AWS), or Google Cloud Secret Manager (Spring Cloud GCP). Unlike Spring Cloud Config, they don't have built-in push notification capabilities, so you need to trigger a configuration change or implement your monitor functionality manually.

4.1.3 Managing secrets with Spring Cloud Config

Managing secrets is a critical task for any software system, and it's dangerous when mistakes are made. So far, we have included passwords either in property files or environment variables, but they were unencrypted in both cases. One of the consequences of not encrypting them is that we can't version-control them safely. We would like to keep everything under version control and use Git repositories as the single sources of truth, which is one of the principles behind the GitOps strategy I'll cover in chapter 15.

The Spring Cloud Config project is well-equipped with features to handle configuration for cloud native applications, including secrets management. The main goal is to include secrets in the property files and put them under version control, which can only be done if they are encrypted.

Spring Cloud Config Server supports encryption and decryption and exposes two dedicated endpoints: /encrypt and /decrypt. Encryption can be based on a symmetric key or asymmetric key pair.

When using a symmetric key, Spring Cloud Config Server decrypts secrets locally and sends them decrypted to the client applications. In production, all communications between applications will happen over HTTPS, so the response sent from Config Service will be encrypted even if the configuration property is not, making this approach secure enough for real-world usage.

You also have the option to send property values encrypted and let the applications themselves decrypt them, but that will require you to configure the symmetric key for all applications. You should also consider that decryption is not a cheap operation to perform.

Spring Cloud Config also supports encryption and decryption through asymmetric keys. This option provides more robust security than the symmetric alternative but it also increases complexity and maintenance costs due to key management tasks. In that case, you might want to consider relying on a dedicated secrets management solution. For example, you can use one of those offered by cloud providers and rely on the Spring Boot integration implemented by Spring Cloud, such as Azure Key Vault (Spring Cloud Azure), AWS Parameter Store or AWS Secrets Manager (Spring Cloud AWS), or Google Cloud Secret Manager (Spring Cloud GCP).

Should you prefer an open source solution, HashiCorp Vault (**www.vaultproject.io**) might be a good fit for you. It's a tool you can use to manage all your credentials, tokens, and certificates, both from a CLI and from a convenient GUI. You can integrate it directly with your Spring Boot applications using the Spring Vault project or add it as an additional backend for Spring Cloud Config Server.

For more information about secrets management in Spring, check out the official documentation for Spring Vault (**https://spring.io/projects/spring-vault**) and Spring Cloud Config (**https://spring.io/projects/spring-cloud-config**).

4.1.4 Disabling Spring Cloud Config

The next section will introduce a different way of configuring Spring Boot applications based on the native functionality provided by Kubernetes through ConfigMaps and Secrets. That's what we're going to use in production.

Even if we're not going to use Config Service anymore in the rest of the book, we'll keep all the work we have done with it so far. However, to make things easier, we'll turn the Spring Cloud Config Client integration off by default.

Open your Catalog Service project (catalog-service), and update the application.yml file to stop importing configuration data from Config Service and disable the Spring Cloud Config Client integration. Everything else will stay the same. Whenever you want to use Spring Cloud Config again, you can enable it with ease (for example, when running the applications on Docker).

Listing 14.1 Disabling Spring Cloud Config in Catalog Service

```
spring:
  config:
    import: ""                    ❶
  cloud:
    config:
      enabled: false              ❷
      uri: http://localhost:8888
      request-connect-timeout: 5000
      request-read-timeout: 5000
      fail-fast: false
      retry:
        max-attempts: 6
        initial-interval: 1000
        max-interval: 2000
        multiplier: 1.1
```

❶ Stops importing configuration data from Config Service

❷ Disables the Spring Cloud Config Client integration

In the next section, you'll use ConfigMaps and Secrets to configure Spring Boot applications instead of the Config Service.

14.2 Using ConfigMaps and Secrets in Kubernetes

The 15-Factor methodology recommends keeping code, configuration, and credentials always separate. Kubernetes fully embraces that principle and defines two APIs to handle configuration and credentials independently: ConfigMaps and Secrets. This section will introduce this new configuration strategy, which is provided natively by Kubernetes.

Spring Boot provides native and flexible support for both ConfigMaps and Secrets. I'll show you how to work with ConfigMaps and their relationships with environment variables, which are still a valid configuration option in Kubernetes. You'll see that Secrets are not really secret, and you'll learn what to do to make them really so. Finally, I'll go through a few options for dealing with configuration changes and propagating them to applications.

Before moving on further, let's set the scene and start a local Kubernetes cluster. Go to your Polar Deployment project (polar-deployment), navigate to the kubernetes/platform/development folder, and run the following command to start a minikube cluster and deploy the backing services used by Polar Bookshop:

```
$ ./create-cluster.sh
```

NOTE If you haven't followed along with the examples implemented in the previous chapters, you can refer to the repository accompanying the book (**https://github.com/ThomasVitale/cloud-native-spring-in-action**) and use the projects in Chapter14/14-begin as a starting point.

The command will take a few minutes to complete. When it's finished, you can verify that all the backing services are ready and available with the following command:

```
$ kubectl get deploy

NAME          READY  UP-TO-DATE  AVAILABLE  AGE
polar-keycloak  1/1    1          1          3m94s
polar-postgres  1/1    1          1          3m94s
polar-rabbitmq  1/1    1          1          3m94s
polar-redis     1/1    1          1          3m94s
polar-ui        1/1    1          1          3m94s
```

Let's start by introducing ConfigMaps.

4.2.1 Configuring Spring Boot with ConfigMaps

In chapter 7, we used environment variables to pass hardcoded configuration to containers running in Kubernetes, but they lack maintainability and structure. ConfigMaps let you store configuration data in a structured, maintainable way. They can be version-controlled together with the rest of your Kubernetes deployment manifests and have the

same nice properties of a dedicated configuration repository, including data persistence, auditing, and accountability.

A *ConfigMap* is "an API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume" (**https://kubernetes.io/docs/concepts/configuration/configmap**).

You can build a ConfigMap starting with a literal key/value pair string, with a file (for example, .properties or .yml), or even with a binary object. When working with Spring Boot applications, the most straightforward way to build a ConfigMap is to start with a property file.

Let's look at an example. In the previous chapters, we configured Catalog Service via environment variables. For better maintainability and structure, let's store some of those values in a ConfigMap.

Open the Catalog Service project (catalog-service), and create a new configmap.yml file in the k8s folder. We'll use it to apply the following configuration, which will overwrite the default values included in the application.yml file packaged with the application:

- Configure a custom greeting.
- Configure the URL for the PostgreSQL data source.
- Configure the URL for Keycloak.

Listing 14.2 Defining a ConfigMap to configure Catalog Service

```
apiVersion: v1                    ❶
kind: ConfigMap                   ❷
metadata:
  name: catalog-config            ❸
  labels:                         ❹
    app: catalog-service
data:                             ❺
  application.yml: |              ❻
    polar:
      greeting: Welcome to the book catalog from Kubernetes!
    spring:
      datasource:
        url: jdbc:postgresql://polar-postgres/polardb_catalog
      security:
        oauth2:
```

```
    resourceserver:
      jwt:
        issuer-uri: http://polar-keycloak/realms/PolarBookshop
```

❶ The API version for ConfigMap objects

❷ The type of object to create

❸ The name of the ConfigMap

❹ A set of labels attached to the ConfigMap

❺ Section containing the configuration data

❻ A key/value pair where the key is the name of a YAML configuration file and the value is its content

Like the other Kubernetes objects we have worked with so far, manifests for ConfigMaps can be applied to a cluster using the Kubernetes CLI. Open a Terminal window, navigate to your Catalog Service project (catalog-service), and run the following command:

```
$ kubectl apply -f k8s/configmap.yml
```

You can verify that the ConfigMap has been created correctly with this command:

```
$ kubectl get cm -l app=catalog-service

NAME             DATA   AGE
catalog-config   1      7s
```

The values stored in a ConfigMap can be used to configure containers running in a few different ways:

- Use a ConfigMap as a configuration data source to pass command-line arguments to the container.
- Use a ConfigMap as a configuration data source to populate environment variables for the container.
- Mount a ConfigMap as a volume in the container.

As you learned in chapter 4 and practiced since then, Spring Boot supports externalized configuration in many ways, including via command-line arguments and environment variables. Passing configuration data as command-line arguments or environment variables to containers has its drawbacks, even if it is stored in a ConfigMap. For example, whenever you add a property to a ConfigMap, you must update the Deployment manifest. When a ConfigMap is changed, the Pod is not informed about it and must be re-created to read the new configuration. Both those issues are solved by mounting ConfigMaps as volumes.

When a ConfigMap is mounted as a volume to a container, it generates two possible outcomes (figure 14.2):

- If the ConfigMap includes an *embedded property file*, mounting it as a volume results in the property file being created in the mounted path. Spring Boot automatically finds and includes any property files located in a `/config` folder either in the same root as the application executable or in a subdirectory, so it's the perfect path for mounting a ConfigMap. You can also specify additional locations to search for property files via the `spring.config.additional-location=<path>` configuration property.
- If the ConfigMap includes *key/value pairs*, mounting it as a volume results in a *config tree* being created in the mounted path. For each key/value pair, a file is created, named like the key and containing the value. Spring Boot supports reading configuration properties from a config tree. You can specify where the config tree should be loaded from via the `spring.config.import=configtree:<path>` property.

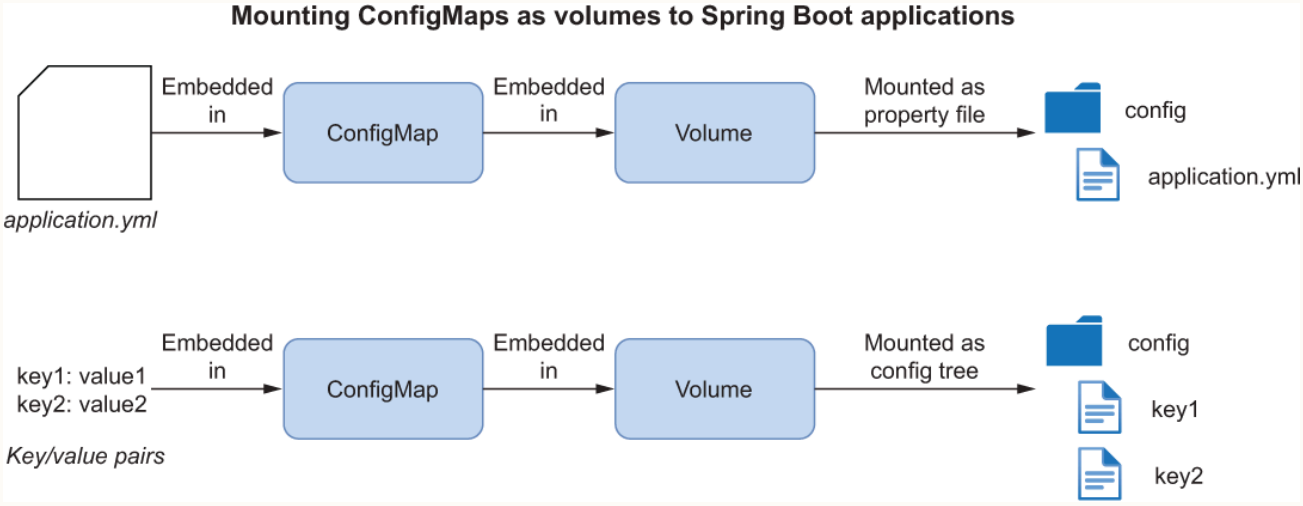


Figure 14.2 ConfigMaps mounted as volumes can be consumed by Spring Boot as property files or as config trees.

When configuring Spring Boot applications, the first option is the most convenient, since it uses the same property file format used for the default configuration inside the appli-

cation. Let's see how we can mount the ConfigMap created earlier into the Catalog Service container.

Open the Catalog Service project (catalog-service), and go to the deployment.yml file in the k8s folder. We need to apply three changes:

- Remove the environment variables for the values we declared in the ConfigMap.
- Declare a volume generated from the catalog-config ConfigMap.
- Specify a volume mount for the catalog-service container to load the ConfigMap as an application.yml file from /workspace/config. The /workspace folder is created and used by Cloud Native Buildpacks to host the application executables, so Spring Boot will automatically look for a /config folder in the same path and load any property files contained within. There's no need to configure additional locations.

Listing 14.3 Mounting a ConfigMap as a volume to the application container

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: catalog-service
  labels:
    app: catalog-service
spec:
  ...
  template:
    ...
    spec:
      containers:
        - name: catalog-service
          image: catalog-service
          imagePullPolicy: IfNotPresent
          ...
          env:                          ❶
            - name: BPL_JVM_THREAD_COUNT
              value: "50"
            - name: SPRING_PROFILES_ACTIVE
              value: testdata
          ...
          volumeMounts:                 ❷
            - name: catalog-config-volume
              mountPath: /workspace/config   ❸
      volumes:                          ❹
        - name: catalog-config-volume   ❺
```

```
      configMap:                    ❻
        name: catalog-config
```

❶ JVM threads and Spring profile are still configured via environment variables.

❷ Mounts the ConfigMap in the container as a volume

❸ Spring Boot will automatically find and include property files from this folder.

❹ Defines volumes for the Pod

❺ The name of the volume

❻ The ConfigMap from which to create a volume

We previously applied the ConfigMap to the cluster. Let's do the same for the Deployment and Service manifests so that we can verify whether Catalog Service is correctly reading configuration data from the ConfigMap.

First, we must package the application as a container image and load it into the cluster. Open a Terminal window, navigate to the root folder of your Catalog Service project (catalog-service), and run the following commands:

```
$ ./gradlew bootBuildImage
$ minikube image load catalog-service --profile polar
```

Now we're ready to deploy the application in the local cluster by applying the Deployment and Service manifests:

```
$ kubectl apply -f k8s/deployment.yml -f k8s/service.yml
```

You can verify when Catalog Service is available and ready to accept requests with this command:

```
$ kubectl get deploy -l app=catalog-service

NAME              READY   UP-TO-DATE   AVAILABLE   AGE
catalog-service   1/1     1            1           21s
```

Internally, Kubernetes uses the liveness and readiness probes we configured in the previous chapter to infer the application's health.

Next, forward traffic from your local machine to the Kubernetes cluster by running the following command:

```
$ kubectl port-forward service/catalog-service 9001:80
Forwarding from 127.0.0.1:9001 -> 9001
Forwarding from [::1]:9001 -> 9001
```

NOTE The process started by the `kubectl port-forward` command will keep running until you explicitly stop it with Ctrl-C.

Now you can call Catalog Service from your local machine on port 9001, and the request will be forwarded to the `Service` object inside the Kubernetes cluster. Open a new Terminal window, and call the root endpoint exposed by the application to verify that the `polar.greeting` value specified in the ConfigMap is used instead of the default one:

```
$ http :9001/
Welcome to the book catalog from Kubernetes!
```

Try also retrieving the books from the catalog to verify that the PostgreSQL URL specified in the ConfigMap is used correctly:

```
$ http :9001/books
```

When you're done testing the application, stop the port-forward process (Ctrl-C) and delete the Kubernetes objects created so far. Open a Terminal window, navigate to your Catalog Service project (catalog-service), and run the following command, but keep the cluster running, since we're going to use it again soon:

```
$ kubectl delete -f k8s
```

ConfigMaps are convenient for providing configuration data to applications running on Kubernetes. But what if we had to pass sensitive data? In the next section, you'll see how to use Secrets in Kubernetes.

4.2.2 Storing sensitive information with Secrets (or not)

The most critical part of configuring applications is managing secret information like passwords, certificates, tokens, and keys. Kubernetes provides a Secret object to hold such data and pass it to containers.

A *Secret* is an API object used to store and manage sensitive information, such as passwords, OAuth tokens, and ssh keys. Pods can consume Secrets as environment variables or configuration files in a volume (**https://kubernetes.io/docs/concepts/configuration/secret**).

What makes this object *secret* is the process used to manage it. By themselves, Secrets are just like ConfigMaps. The only difference is that data in a Secret is usually Base64-encoded, a technical choice made to support binary files. Any Base64-encoded object can be decoded in a very straightforward way. It's a common mistake to think that Base64 is a kind of encryption. If you remember only one thing about Secrets, make it the following: *Secrets are not secret!*

The configuration we have been using to run Polar Bookshop on a local Kubernetes cluster relies on the same default credentials used in development, so we won't need Secrets yet. We'll start using them in the next chapter when deploying applications in production. For now, I want to show you how to create Secrets. Then I'll go through some options you have for ensuring that they are adequately protected.

One way of creating a Secret is using the Kubernetes CLI with an imperative approach. Open a Terminal window and generate a test-credentials Secret object for some fictitious test credentials (user/password).

```
$ kubectl create secret generic \          ❶
  test-credentials \                       ❷
  --from-literal=test.username=user \      ❸
  --from-literal=test.password=password    ❹
```

❶ Creates a generic secret with Base64-encoded values

❷ The name of the Secret

❸ Adds a secret value for the test username

❹ Adds a secret value for the test password

We can verify that the Secret has been created successfully with the following command:

```
$ kubectl get secret test-credentials

NAME              TYPE    DATA   AGE
test-credentials  Opaque  2      73s
```

We can also retrieve the internal representation of the Secret in the familiar YAML format with the following command:

```
$ kubectl get secret test-credentials -o yaml

apiVersion: v1                ❶
kind: Secret                  ❷
metadata:
  name: test-credentials      ❸
type: Opaque
data:                         ❹
  test.username: dXNlcg==
  test.password: cGFzc3dvcmQ=
```

❶ The API version for Secret objects

❷ The type of object to create

❸ The name of the Secret

❹ Section containing the secret data with Base64-encoded values

Note that I rearranged the preceding YAML to increase its readability and omitted additional fields that are not relevant to our discussion.

I want to repeat this: Secrets are not secret! I can decode the value stored in the `test-credentials` Secret with a simple command:

```
$ echo 'cGFzc3dvcmQ=' | base64 --decode
password
```

Like ConfigMaps, Secrets can be passed to a container as environment variables or through a volume mount. In the second case, you can mount them as property files or config trees. For example, the test-credentials Secret would be mounted as a config tree because it's composed of key/value pairs rather than a file.

Since Secrets are not encrypted, we can't include them in a version control system. It's up to the platform engineers to ensure that Secrets are adequately protected. For example, Kubernetes could be configured to store Secrets in its internal *etcd* storage encrypted. That would help ensure security at rest, but it doesn't solve the problem of managing them in a version control system.

Bitnami introduced a project called *Sealed Secrets* (**https://github.com/bitnami-labs/sealed-secrets**), aimed at encrypting Secrets and putting them under version control. First you would generate an encrypted SealedSecret object, starting from literal values, similar to what we did for the plain Secret. Then you would include that in your repository and safely put it under version control. When the SealedSecret manifest is applied to a Kubernetes cluster, the Sealed Secrets controller decrypts its content and generates a standard Secret object that can be used within a Pod.

What if your secrets are stored in a dedicated backend like HashiCorp Vault or Azure Key Vault? In that case, you can use a project like *External Secrets* (**https://github.com/external-secrets/kubernetes-external-secrets**). As you can guess from its name, this project lets you generate a Secret from an external source. The ExternalSecret object would be safe to store in your repository and put under version control. When the ExternalSecret manifest is applied to a Kubernetes cluster, the External Secrets controller fetches the value from the configured external source and generates a standard Secret object that can be used within a Pod.

NOTE If you're interested in learning more about how to secure Kubernetes Secrets, you can check out chapter 7 of *GitOps and Kubernetes* by Billy Yuen, Alexander Matyushentsev, Todd Ekenstam, and Jesse Suen (Manning, 2021) and *Kubernetes Secrets Management* by Alex Soto Bueno and Andrew Block (Manning, 2022). I won't provide more information here, since this is usually a task for the platform team, not developers.

When we start using ConfigMaps and Secrets, we must decide which policy to use to update configuration data and how to make applications use the new values. That's the topic of the next section.

4.2.3 Refreshing configuration at runtime with Spring Cloud Kubernetes

When using an external configuration service, you'll probably want a mechanism to reload the applications when configuration changes. For example, when using Spring Cloud Config, we can implement such a mechanism with Spring Cloud Bus.

In Kubernetes, we need a different approach. When you update a ConfigMap or a Secret, Kubernetes takes care of providing containers with the new versions when they're mounted as volumes. If you use environment variables, they will not be replaced with the new values. That's why we usually prefer the volume solution.

The updated ConfigMaps or Secrets are provided to the Pod when they're mounted as volumes, but it's up to the specific application to refresh the configuration. By default, Spring Boot applications read configuration data only at startup time. There are three main options for refreshing configuration when it's provided through ConfigMaps and Secrets:

- *Rolling restart*—Changing a ConfigMap or a Secret can be followed by a rolling restart of all the Pods affected, making the applications reload all the configuration data. With this option, Kubernetes Pods would remain immutable.
- *Spring Cloud Kubernetes Configuration Watcher*—Spring Cloud Kubernetes provides a Kubernetes controller called Configuration Watcher that monitors ConfigMaps and Secrets mounted as volumes to Spring Boot applications. Leveraging the Spring Boot Actuator's `/actuator/refresh` endpoint or Spring Cloud Bus, when any of the ConfigMaps or Secrets is updated, the Configuration Watcher will trigger a configuration refresh for the affected applications.
- *Spring Cloud Kubernetes Config Server*—Spring Cloud Kubernetes provides a configuration server with support for using ConfigMaps and Secrets as one of the configuration data source options for Spring Cloud Config. You could use such a server to load configuration from both a Git repository and Kubernetes objects, with the possibility of using the same configuration refresh mechanism for both.

For Polar Bookshop, we'll use the first option and rely on Kustomize to trigger a restart of the applications whenever a new change is applied to a ConfigMap or a Secret. I'll describe that strategy further in the next section of the chapter. Here we'll focus on the features offered by Spring Cloud Kubernetes and its subprojects.

Spring Cloud Kubernetes (**https://spring.io/projects/spring-cloud-kubernetes**) is an exciting project that provides Spring Boot integration with the Kubernetes API. Its original goal was to make it easier to transition from a microservices architecture based on

Spring Cloud to Kubernetes. It provides an implementation for standard Spring Cloud interfaces used for service discovery and load balancing to integrate with Kubernetes, and it adds support for loading configuration from ConfigMaps and Secrets.

If you work on a greenfield project, you don't need Spring Cloud Kubernetes. Kubernetes provides service discovery and load balancing natively, as you experienced in chapter 7. Furthermore, Spring Boot supports configuration via ConfigMaps and Secrets natively, so there's no need for Spring Cloud Kubernetes, even in this case.

When migrating a brownfield project to Kubernetes, and it uses libraries like Spring Cloud Netflix Eureka for service discovery and Spring Cloud Netflix Ribbon or Spring Cloud Load Balancer for load balancing, you might use Spring Cloud Kubernetes for a smoother transition. However, I would recommend refactoring your code to leverage the native service discovery and load-balancing features from Kubernetes rather than adding Spring Cloud Kubernetes to your project.

The main reason why I recommend not using Spring Cloud Kubernetes in standard applications is that it requires access to the Kubernetes API Server to manage Pods, Services, ConfigMaps, and Secrets. Besides the security concerns related to granting applications access to the Kubernetes internal objects, it would also couple the applications to Kubernetes unnecessarily and affect the maintainability of the solution.

When does it make sense to use Spring Cloud Kubernetes? As one example, Spring Cloud Gateway could be enhanced with Spring Cloud Kubernetes to get more control over service discovery and load balancing, including automatic registration of new routes based on Services metadata and the choice of load-balancing strategy. In this case, you could rely on the Spring Cloud Kubernetes Discovery Server component, limiting the need for Kubernetes API access to the discovery server.

Spring Cloud Kubernetes really shines when it comes to implementing Kubernetes controller applications to accomplish administrative tasks within the cluster. For example, you could implement a controller that monitors when ConfigMaps or Secrets change and then triggers a configuration refresh on the application using them. As a matter of fact, the Spring team used Spring Cloud Kubernetes to build a controller that does precisely that: the Configuration Watcher.

NOTE Spring Cloud Kubernetes Configuration Watcher is available as a container image on Docker Hub. If you'd like to know more about how it works and how to deploy it, you can refer to the official documentation (**https://spring.io/projects/spring-cloud-kubernetes**).

Besides the Configuration Watcher, Spring Cloud Kubernetes provides other convenient off-the-shelf applications for addressing common concerns of distributed systems in Kubernetes. One of them is a configuration server built on top of Spring Cloud Config and extending its functionality to support reading configuration data from ConfigMaps and Secrets. It's called Spring Cloud Kubernetes Config Server.

You can use this application directly (the container image is published on Docker Hub) and deploy it on Kubernetes following the instructions provided in the official documentation (**https://spring.io/projects/spring-cloud-kubernetes**).

As an alternative, you can use its source code on GitHub as a foundation to build your own Kubernetes-aware configuration server. For example, as I explained earlier in this chapter, you might want to protect it via HTTP Basic authentication. In that case, you could use your experience working with Spring Cloud Config and build an enhanced version of Config Service for Polar Bookshop on top of Spring Cloud Kubernetes Config Server.

In the next section, I will introduce Kustomize for managing deployment configurations in Kubernetes.

14.3 Configuration management with Kustomize

Kubernetes provides many useful features for running cloud native applications. Still, it requires writing several YAML manifests, which are sometimes redundant and not easy to manage in a real-world scenario. After collecting the multiple manifests needed to deploy an application, we are faced with additional challenges. How can we change the values in a ConfigMap depending on the environment? How can we change the container image version? What about Secrets and volumes? Is it possible to update the health probe's configuration?

Many tools have been introduced in the last few years to improve how we configure and deploy workloads in Kubernetes. For the Polar Bookshop system, we would like a tool that lets us handle multiple Kubernetes manifests as a single entity and customize parts of the configuration depending on the environment where the application is deployed.

Kustomize (**https://kustomize.io**) is a declarative tool that helps configure deployments for different environments via a layering approach. It produces standard Kubernetes manifests, and it's built natively in the Kubernetes CLI (kubectl), so you don't need to install anything else.

NOTE Other popular options for managing deployment configuration in Kubernetes are ytt from the Carvel suite (**https://carvel.dev/ytt**) and Helm (**https://helm.sh**).

This section will show you the key features offered by Kustomize. First you'll see how to *compose* related Kubernetes manifests and handle them as a single unit. Then I'll show you how Kustomize can generate a ConfigMap for you from a property file. Finally, I'll guide you through a series of customizations that we'll apply to the base manifests before deploying workloads in a staging environment. The next chapter will expand on that and cover the production scenario.

Before moving on, make sure you still have your local minikube cluster up and running and that the Polar Bookshop backing services have been deployed correctly. If you don't, run `./create-cluster.sh` from polar-deployment/kubernetes/platform/ development.

NOTE The platform services are exposed only within the cluster. If you want to access any of them from your local machine, you can use the port-forwarding feature you learned about in chapter 7. You can either leverage the GUI provided by Octant or use the CLI (`kubectl port-forward service/polar-postgres 5432:5432`).

Now that we have all the backing services available, let's see how we can manage and configure a Spring Boot application using Kustomize.

14.3.1 Using Kustomize to manage and configure Spring Boot applications

So far, we've been deploying applications to Kubernetes by applying multiple Kubernetes manifests. For example, deploying Catalog Service requires applying the ConfigMap, Deployment, and Service manifests to the cluster. When using Kustomize, the first step is composing related manifests together so that we can handle them as a single unit. Kustomize does that via a `Kustomization` resource. In the end, we want to let Kustomize manage, process, and generate Kubernetes manifests for us.

Let's see how it works. Open your Catalog Service project (catalog-service) and create a kustomization.yml file inside the k8s folder. It will be the entry point for Kustomize.

We'll first instruct Kustomize about which Kubernetes manifests it should use as a foundation for future customizations. For now, we'll use the existing Deployment and Service manifests.

Listing 14.4 Defining the base Kubernetes manifests for Kustomize

```
apiVersion: kustomize.config.k8s.io/v1beta1     ❶
kind: Kustomization                             ❷

resources:                                      ❸
  - deployment.yml
  - service.yml
```

❶ The API version for Kustomize

❷ The kind of resource defined by the manifest

❸ Kubernetes manifests that Kustomize should manage and process

You might be wondering why we didn't include the ConfigMap. I'm glad you asked! We could have included the configmap.yml file we created earlier in the chapter, but Kustomize offers a better way. Instead of referencing a ConfigMap directly, we can provide a property file and let Kustomize use it to generate a ConfigMap. Let's see how it works.

For starters, let's move the body of the ConfigMap we created previously (configmap.yml) to a new application.yml file within the k8s folder.

Listing 14.5 Configuration properties provided via a ConfigMap

```
polar:
  greeting: Welcome to the book catalog from Kubernetes!
spring:
  datasource:
    url: jdbc:postgresql://polar-postgres/polardb_catalog
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://polar-keycloak/realms/PolarBookshop
```

Then delete the configmap.yml file. We won't need it anymore. Finally, update the kustomization.yml file to generate a catalog-config ConfigMap starting from the application.yml file we just created.

Listing 14.6 Getting Kustomize to generate a ConfigMap from a property file

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - deployment.yml
  - service.yml

configMapGenerator:          (1)
  - name: catalog-config
    files:                   (2)
      - application.yml
    options:
      labels:                (3)
        app: catalog-service
```

(1) The section containing information to generate ConfigMaps

(2) Uses a property file as the source for a ConfigMap

(3) Defines the labels to assign to the generated ConfigMap

NOTE In a similar way, Kustomize can also generate Secrets starting with literal values or files.

Let's pause for a moment and verify that what we have done so far works correctly. Your local cluster should already have your Catalog Service container image from before. If that's not the case, build the container image (`./gradlew bootBuildImage`), and load it into minikube (`minikube image load catalog-service --profile polar`).

Next, open a Terminal window, navigate to your Catalog Service project (catalog-service), and deploy the application using the familiar Kubernetes CLI. When applying standard Kubernetes manifests, we use the `-f` flag. When applying a Kustomization, we use the `-k` flag:

```
$ kubectl apply -k k8s
```

The final result should be the same as we got earlier when applying the Kubernetes manifests directly, but this time Kustomize handled everything via a Kustomization resource.

To complete the verification, use the port-forwarding strategy to expose the Catalog Service application to your local machine (kubectl port-forward service/catalog-service 9001:80). Then open a new Terminal window, and ensure that the root endpoint returns the message configured via the ConfigMap generated by Kustomize:

```
$ http :9001/
Welcome to the book catalog from Kubernetes!
```

ConfigMaps and Secrets generated by Kustomize are named with a unique suffix (a *hash*) when they're deployed. You can verify the actual name assigned to the catalog-config ConfigMap with the following command:

```
$ kubectl get cm -l app=catalog-service

NAME                     DATA   AGE
catalog-config-btcmff5d78  1    7m58s
```

Every time you update the input to the generators, Kustomize creates a new manifest with a different hash, which triggers a *rolling restart* of the containers where the updated ConfigMaps or Secrets are mounted as volumes. That is a highly convenient way to achieve an automated configuration refresh without implementing or configuring any additional components.

Let's verify that it's true. First, update the value for the polar.greeting property in the application.yml file used by Kustomize to generate the ConfigMap.

Listing 14.7 Updating the configuration input to the ConfigMap generator

```
polar:
  greeting: Welcome to the book catalog from a development
  ➥ Kubernetes environment!
...
```

Then apply the Kustomization again (kubectl apply -k k8s). Kustomize will generate a new ConfigMap with a different suffix hash, triggering a rolling restart of all the Catalog Service instances. In this case there's only one instance running. In production there will be more. The fact that the instances are restarted one at a time means that the

update happens with zero downtime, which is what we aim for in the cloud. The Catalog Service root endpoint should now return the new message:

```
$ http :9001/
Welcome to the book catalog from a development Kubernetes environment!
```

If you're curious, you could compare this result with what would happen when updating a ConfigMap without Kustomize. Kubernetes would update the volume mounted to the Catalog Service container, but the application would not be restarted and would still return the old value.

NOTE Depending on your requirements, you might need to avoid a rolling restart and have the applications reload their configuration at runtime. In that case, you can disable the hash suffix strategy with the `disableNameSuffixHash: true` generator option and perhaps rely on something like Spring Cloud Kubernetes Configuration Watcher to notify the applications whenever a ConfigMap or Secret is changed.

When you're done experimenting with the Kustomize setup, you can stop the port-forwarding process (Ctrl-C) and undeploy Catalog Service (`kubectl delete -k k8s`).

Since we moved from plain Kubernetes manifests to Kustomize, we still need to update a couple of things. In chapter 7, we used Tilt to achieve a better development workflow when working locally on Kubernetes. Tilt supports Kustomize, so we can configure it to deploy applications via a `Kustomization` resource rather than via plain Kubernetes manifests. Go ahead and update the Tiltfile in your Catalog Service project as follows.

Listing 14.8 Configuring Tilt to deploy Catalog Service using Kustomize

```
custom_build(
   ref = 'catalog-service',
   command = './gradlew bootBuildImage --imageName $EXPECTED_REF',
   deps = ['build.gradle', 'src']
)

k8s_yaml(kustomize('k8s'))     (1)

k8s_resource('catalog-service', port_forwards=['9001'])
```

(1) Runs the application from the Kustomization located in the k8s folder

Finally, we need to update the manifest validation step in the commit stage workflow for Catalog Service, or it will fail the next time we push changes to GitHub. In your Catalog Service project, open the commit-stage.yml file (.github/workflows) and update it as follows.

Listing 14.9 Using Kubeval to validate the manifests generated by Kustomize

```
name: Commit Stage
on: push
...
jobs:
  build:
    name: Build and Test
    ...
    steps:
      ...
      - name: Validate Kubernetes manifests
        uses: stefanprodan/kube-tools@v1
        with:
          kubectl: 1.24.3
          kubeval: 0.16.1
          command: |
            kustomize build k8s | kubeval --strict -      (1)
```

(1) Uses Kustomize to generate the manifests and then validates them with Kubeval

So far, the most significant benefit we got from Kustomize is the automatic rolling restart of applications when a ConfigMap or Secret is updated. In the next section, you'll learn more about Kustomize and explore its powerful features for managing different Kubernetes configurations depending on the deployment environment.

4.3.2 Managing Kubernetes configuration for multiple environments with Kustomize

During development we followed the 15-Factor methodology and externalized the configuration for each aspect of an application that could change between deployments in different environments. You saw how to use property files, environment variables, configuration services, and ConfigMaps. I also showed you how to use Spring profiles to customize the application configuration based on the deployment environment. Now we need to take a step further and define a strategy to customize the entire deployment configuration depending on where we deploy an application.

In the previous section, you learned how to compose and process Kubernetes manifests together via a Kustomization resource. For each environment, we can specify *patches* to apply changes or additional configurations on top of those basic manifests. All the customization steps you'll see in this section will be applied without changing anything in the application source code but using the same release artifacts produced earlier. That's quite a powerful concept and one of the main features of cloud native applications.

The Kustomize approach to configuration customization is based on the concepts of *bases* and *overlays*. The k8s folder we created in the Catalog Service project can be considered a *base* : a directory with a kustomization.yml file that combines Kubernetes manifests and customizations. An *overlay* is another directory with a kustomization.yml file. What makes it special is that it defines customizations in relation to one or more bases and combines them. Starting from the same base, you can specify an overlay for each deployment environment (such as development, test, staging, and production).

As shown in figure 14.3, each Kustomization includes a kustomization.yml file. The one acting as the *base* composes together several Kubernetes resources like Deployments, Services, and ConfigMaps. Also, it's not aware of the overlays, so it's completely independent of them. The *overlays* use one or more *bases* as a foundation and provide additional configuration via patches.

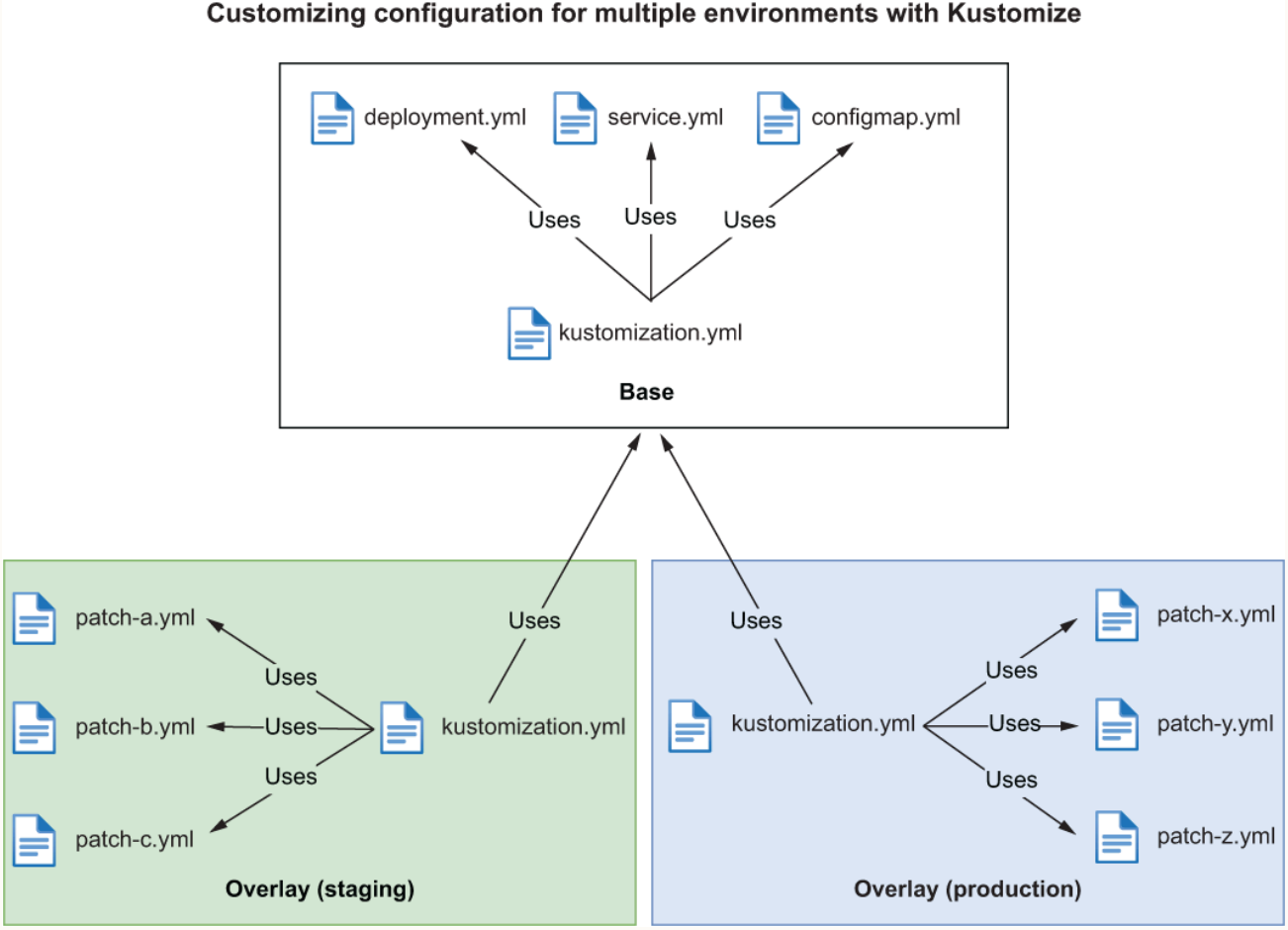


Figure 14.3 Kustomize bases can be used as the foundation for further customizations (overlays) depending on the deployment environment.

Bases and overlays can be defined either in the same repository or different ones. For the Polar Bookshop system, we'll use the k8s folder in each application project as a base and define overlays in the polar-deployment repository. Similar to what you learned in chapter 3 about application codebases, you can decide whether to keep your deployment configuration in the same repository as your application or not. I decided to go for a separate repository for a few reasons:

- It makes it possible to control the deployment of all the system components from a single place.
- It allows focused version-control, auditing, and compliance checks before deploying anything to production.
- It fits the GitOps approach, where delivery and deployment tasks are decoupled.

As an example, figure 14.4 shows how the Kustomize manifests could be structured in the case of Catalog Service, having bases and overlays in two separate repositories.

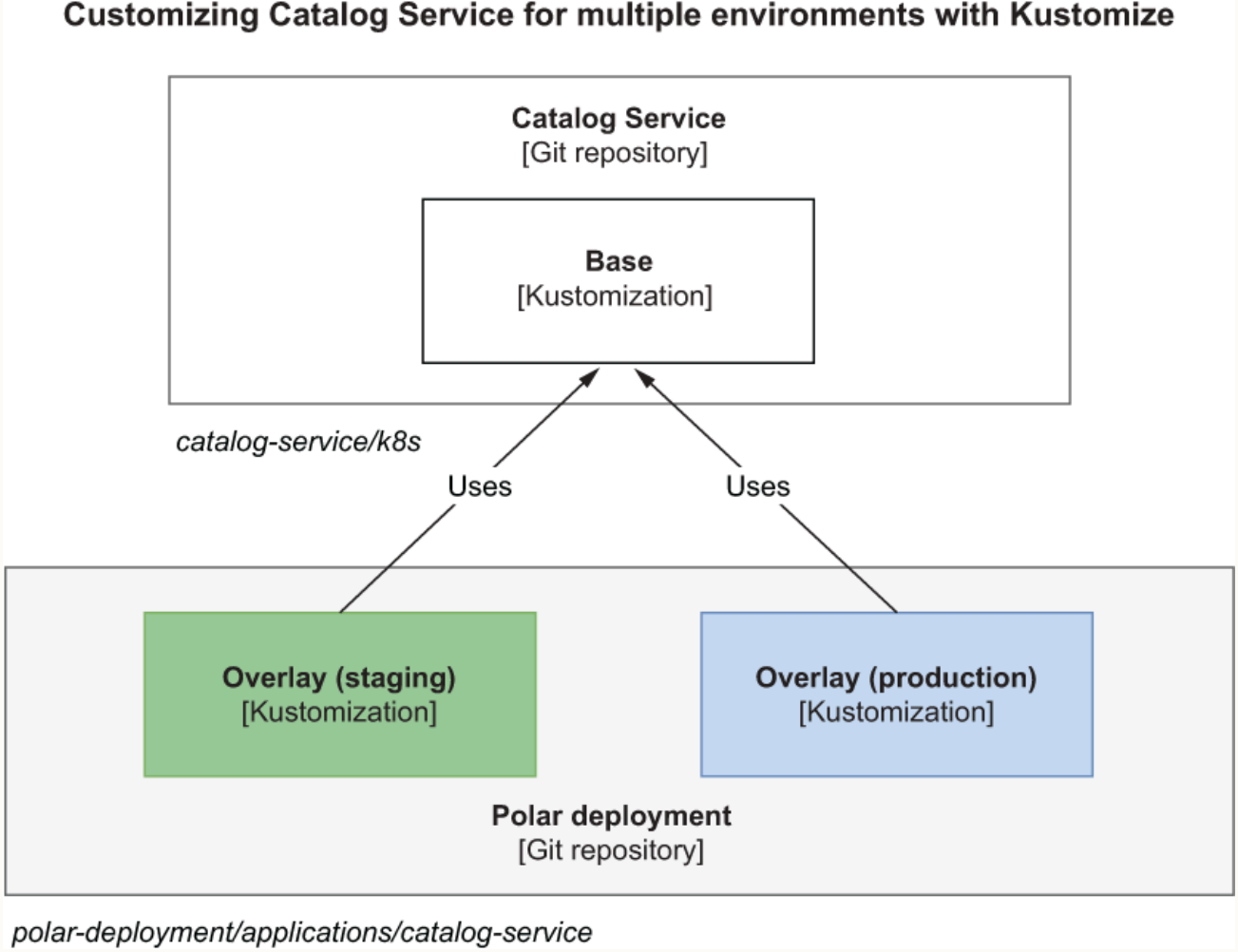


Figure 14.4 Kustomize bases and overlays can be stored in the same repository or two separate ones. Overlays can be used to customize deployments for different environments.

Another decision to make is whether to keep the base Kubernetes manifests together with the application source code or move them to the deployment repository. I decided to go with the first approach for the Polar Bookshop example, similar to what we did with the default configuration properties. One of the benefits is that it makes it simple to run each application on a local Kubernetes cluster during development, either directly or using Tilt. Depending on your requirements, you might decide to use one approach or the other. Both are valid and used in real-world scenarios.

Patches vs. templates

Kustomize's approach to customizing configuration is based on applying patches. It's quite the opposite of how Helm works (**https://helm.sh**). Helm requires you to template every part of a manifest that you would like to change (resulting in non-valid YAML). After that, you can provide different values for those templates in each environment. If a field is not templated, you can't customize its value. For that reason, it's not rare to use Helm and Kustomize in sequence, overcoming each other's shortcomings. Both approaches have pros and cons.

In this book I decided to use Kustomize because it's natively available in the Kubernetes CLI, it works with valid YAML files, and it's purely declarative. Helm is more powerful and can also handle complex application rollouts and upgrades that Kubernetes doesn't support natively. On the other hand, it has a steep learning curve, its templating solution has a few drawbacks, and it's not declarative.

Another option is ytt from the Carvel suite (**https://carvel.dev/ytt**). It provides a superior experience, with support for both patches and templates, it works with valid YAML files, and its templating strategy is more robust. It takes a bit more effort to get familiar with ytt than Kustomize, but it's worth the effort. Because it treats YAML as a first-class citizen, ytt can be used to configure and customize any YAML file, even outside Kubernetes. Do you use GitHub Actions workflows? Ansible playbooks? Jenkins pipelines? You can use ytt in all those scenarios.

Let's consider Catalog Service. We already have the base deployment configuration composed with Kustomize. It's located within the project repository in a dedicated folder (catalog-service/k8s). Now let's define an overlay to customize the deployment for staging.

4.3.3 Defining a configuration overlay for staging

In the previous sections, we used Kustomize to manage the configuration of Catalog Service in a local development environment. Those manifests will represent the *base* for multiple customizations applied for each environment as *overlays*. Since we'll define overlays in the polar-deployment repository while the base is in the catalog-service repository, all the Catalog Service manifests must be available in the main remote branch. If you haven't done so yet, push all the changes applied to your Catalog Service project so far to the remote repository on GitHub.

NOTE As I explained in chapter 2, I expect you have created a different repository on GitHub for each project in the Polar Bookshop system. In this chapter we're working only with the polar-deployment and catalog-service repositories, but you should have also created repositories for edge-service, order-service, and dispatcher-service.

As anticipated, we'll store any configuration overlay in the polar-deployment repository. In this section and the following ones, we'll define an overlay for the staging

environment. The next chapter will cover production.

Go ahead and create a new kubernetes/applications folder in your polar-deployment repository. We'll use it to keep the customizations for all the applications in the Polar Bookshop system. In the newly created path, add a catalog-service folder that will contain any overlay for customizing the deployment of Catalog Service in different environments. In particular, we'll want to prepare the deployment in staging, so create a "staging" folder for Catalog Service.

Any customization (base or overlay) requires a kustomization.yml file. Let's create one for the staging overlay of Catalog Service (polar-deployment/kubernetes/applications/catalog-service/staging). The first thing to configure is a reference to the base manifests.

If you've followed along, you should have your Catalog Service source code tracked in a catalog-service repository on GitHub. A reference to a remote base needs to point to the folder containing the kustomization.yml file, which is k8s in our case. Also, we should refer to a specific tag or digest for the version we want to deploy. We'll talk about release strategies and versioning in the next chapter, so we'll simply point to the main branch for now. The final URL should be something like github.com/<your_github_username>/catalog-service/k8s?ref=main. For example, in my case, it would be github.com/polarbookshop/catalog-service/k8s?ref=main.

Listing 14.10 Defining an overlay for staging on top of a remote base

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:                                    (1)
  - github.com/<your_github_username>/catalog-service/k8s?ref=main
```

(1) Uses the manifests in your Catalog Service repo on GitHub as the base for further customizations

NOTE I'll assume that all the GitHub repositories you created for Polar Bookshop are publicly accessible. If that's not the case, you can go to the specific repository page on GitHub and access the Settings section for that repository. Then scroll to the bottom of the settings page, and make the package public by clicking the Change Visibility button.

We could now deploy Catalog Service from the staging overlay using the Kubernetes CLI, but the result wouldn't be different than using the base directly. Let's start applying some customizations specifically for staging deployments.

4.3.4 Customizing environment variables

The first customization we could apply is an environment variable to activate the staging Spring profile for Catalog Service. Most customizations can be applied via patches following a merge strategy. Much like Git merges changes from different branches, Kustomize produces final Kubernetes manifests with changes coming from different Kustomization files (one or more bases and an overlay).

A best practice when defining Kustomize patches is to keep them small and focused. To customize environment variables, create a patch-env.yml file within the staging overlay for Catalog Service (kubernetes/applications/catalog-service/staging). We need to specify some contextual information so Kustomize can figure out where to apply the patch and how to merge the changes. When the patch is for customizing a container, Kustomize requires us to specify the kind and name of the Kubernetes resource (that is, Deployment) and the name of the container. This customization option is called a *strategic merge patch*.

Listing 14.11 A patch for customizing environment variables

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: catalog-service
spec:
  template:
    spec:
      containers:
        - name: catalog-service
          env:
            - name: SPRING_PROFILES_ACTIVE      (1)
              value: prod
```

(1) Defines which Spring profiles should be activated

Next we need to instruct Kustomize to apply the patch. In the kustomization.yml file for the staging overlay of Catalog Service, list the patch-env.yml file as follows.

Listing 14.12 Getting Kustomize to apply the patch for environment variables

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - github.com/<your_github_username>/catalog-service/k8s?ref=main

 patchesStrategicMerge:      ❶
  - patch-env.yml            ❷
```

❶ Section containing the list of patches to apply to the base manifests according to the strategic merge strategy

❷ The patch for customizing the environment variables passed to the Catalog Service container

You can use this same approach to customize many aspects of a Deployment, such as the number of replicas, liveness probe, readiness probe, graceful shutdown timeout, environment variables, volumes, and more. In the next section, I'll show you how to customize ConfigMaps.

4.3.5 Customizing ConfigMaps

The base Kustomization for Catalog Service instructs Kustomize to generate a catalog-config ConfigMap starting from an application.yml file. To customize the values in that ConfigMap, we have two main options: replace the entire ConfigMap or overwrite only the values that should be different in staging. In this second case, we could generally rely on some advanced Kustomize patching strategy to overwrite specific values in the ConfigMap.

When working with Spring Boot, we can take advantage of the power of Spring profiles. Instead of updating values in the existing ConfigMap, we can add an application-staging.yml file, which we know takes precedence over application.yml when the staging profile is active. The final result will be a ConfigMap containing both files.

First, let's create an application-staging.yml file within the staging overlay for Catalog Service. We'll use this property file to define a different value for the polar .greeting property. Since we'll use the same minikube cluster from earlier as the staging environ-

ment, URLs to backing services and credentials will be the same as in the development environment. In a real-world scenario, this stage would involve more customizations.

Listing 14.13 Staging-specific configuration for Catalog Service

```
polar:
  greeting: Welcome to the book catalog from a staging
↪Kubernetes environment!
```

Next we can rely on the ConfigMap Generator provided by Kustomize to combine the application-staging.yml file (defined in the staging overlay) with the application.yml file (defined in the base Kustomization) within the same catalog-config ConfigMap. Go ahead and update the kustomization.yml file for the staging overlay as follows.

Listing 14.14 Merging property files within the same ConfigMap

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - github.com/<your_github_username>/catalog-service/k8s?ref=main

patchesStrategicMerge:
  - patch-env.yml

configMapGenerator:
  - behavior: merge                    (1)
    files:
      - application-staging.yml        (2)
    name: catalog-config               (3)
```

(1) Merges this ConfigMap with the one defined in the base Kustomization

(2) The additional property file added to the ConfigMap

(3) The same ConfigMap name used in the base Kustomization

That's it for ConfigMaps. The following section will cover how you can configure which image name and version to deploy.

4.3.6 Customizing image name and version

The base Deployment manifest defined in the Catalog Service repository (catalog-service/k8s/deployment.yml) is configured to use a local container image and doesn't specify a version number (which means the latest tag is used). That's convenient in the development phase, but it doesn't work for other deployment environments.

If you followed along, you should have your Catalog Service source code tracked in a catalog-service repository on GitHub and a ghcr.io/<your_github_username>/catalog-service:latest container image published to GitHub Container Registry (as per the Commit Stage workflow). The next chapter will cover release strategies and versioning. Until then, we'll still use the latest tag. Regarding the image name, though, it's time to start pulling container images from the registry rather than using the local ones.

NOTE Images published to GitHub Container Registry will have the same visibility as the related GitHub code repository. I'll assume that all the images we build for Polar Bookshop are publicly accessible via the GitHub Container Registry. If that's not the case, you can go to the specific repository page on GitHub and access the Packages section for that repository. Then select Package Settings from the sidebar menu, scroll to the bottom of the settings page, and make the package public by clicking the Change Visibility button.

Similar to what we've done for environment variables, we could use a patch to change the image that's used by the Catalog Service Deployment resource. Since it's a very common customization and would need to be changed every time we deliver a new version of our applications, however, Kustomize provides a more convenient way to declare which image name and version we want to use for each container. Furthermore, we can either update the kustomization.yml file directly or rely on the Kustomize CLI (installed as part of the Kubernetes CLI). Let's try the latter.

Open a Terminal window, navigate to the staging overlay for Catalog Service (kubernetes/applications/catalog-service/staging), and run the following command to define which image and version to use for the catalog-service container. Remember to replace <your_github_username> with your GitHub username in lowercase:

```
$ kustomize edit set image \
    catalog-service=ghcr.io/<your_github_username>/catalog-service:latest
```

This command will automatically update the kustomization.yml file with the new configuration, as you can see in the following listing.

Listing 14.15 Configuring the image name and version for the container

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - github.com/<your_github_username>/catalog-service/k8s?ref=main

patchesStrategicMerge:
  - patch-env.yml

configMapGenerator:
  - behavior: merge
    files:
      - application-staging.yml
    name: catalog-config

images:
  - name: catalog-service                                    (1)
    newName: ghcr.io/<your_github_username>/catalog-service  (2)
    newTag: latest                                           (3)
```

(1) The name of the container as defined in the Deployment manifest

(2) The new image name for the container (with your GitHub username in lowercase)

(3) The new tag for the container

In the next section, I'll show you how to configure the number of replicas to deploy.

14.3.7 Customizing the number of replicas

Cloud native applications should be highly available, and Catalog Service is not. So far we've been deploying a single application instance. What happens if it crashes or becomes momentarily unavailable due to a high workload? We would not be able to use the application anymore. Not very resilient, is it? Among other things, a staging environment is a good target for performance and availability tests. At a minimum, we should have two instances running. Kustomize provides a convenient way to update the number of replicas for a given Pod.

Open the kustomization.yml file in the staging overlay for Catalog Service (kubernetes/applications/catalog-service/staging) and configure two replicas for the application.

Listing 14.16 Configuring replicas for the Catalog Service container

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - github.com/<your_github_username>/catalog-service/k8s?ref=main

patchesStrategicMerge:
  - patch-env.yml

configMapGenerator:
  - behavior: merge
    files:
      - application-staging.yml
    name: catalog-config

images:
  - name: catalog-service
    newName: ghcr.io/<your_github_username>/catalog-service
    newTag: latest

replicas:
  - name: catalog-service     (1)
    count: 2                  (2)
```

(1) The name of the Deployment for which to define the number of replicas

(2) The number of replicas

It's finally time to deploy Catalog Service and test the configuration provided by the staging overlay. For simplicity, we'll use the same minikube local cluster we have been using so far as the staging environment. If you still have your minikube cluster up and running from before, you're good to go. Otherwise, you can start it by running ./create-cluster.sh from polar-deployment/kubernetes/platform/development. The script will spin up a Kubernetes cluster and deploy the backing services required by Polar Bookshop.

Then open a Terminal window, navigate to the staging overlay folder for Catalog Service (applications/catalog-service/staging), and run the following command to deploy the

application via Kustomize:

```
$ kubectl apply -k .
```

You can monitor the operation's result via the Kubernetes CLI (`kubectl get pod -l app=catalog-service`) or the Octant GUI (refer to chapter 7 for more information). Once the applications are available and ready, we can check the application logs using the CLI:

```
$ kubectl logs deployment/catalog-service
```

One of the first Spring Boot log events will tell you that the `staging` profile is enabled, just like we configured in the staging overlay via a patch.

The application is not exposed outside the cluster, but you can use the port-forwarding functionality to forward traffic from your local environment on port 9001 to the Service running in the cluster on port 80:

```
$ kubectl port-forward service/catalog-service 9001:80
```

Next, open a new Terminal window and call the application's root endpoint:

```
$ http :9001
Welcome to the book catalog from a staging Kubernetes environment!
```

The result is the customized message we defined in the application-staging.yml file for the `polar.greeting` property. That's exactly what we were expecting.

NOTE It's worth noticing that if you send a `GET` request to `:9001/books`, you'll get an empty list. In staging, we haven't enabled the `testdata` profile controlling the generation of books at startup time. We want that only in a development or test environment.

The last customization we applied to the staging overlay was the number of replicas to deploy. Let's verify that with the following command:

```
$ kubectl get pod -l app=catalog-service

NAME                       READY   STATUS    RESTARTS   AGE
catalog-service-6c5fc7b955-9kvgf   1/1     Running   0          3m94s
catalog-service-6c5fc7b955-n7rgl   1/1     Running   0          3m94s
```

Kubernetes is designed to ensure the availability of each application. If enough resources are available, it will try to deploy the two replicas on two different nodes. If one node crashes, the application will still be available on the other one. At the same time, Kubernetes takes care of deploying the second instance somewhere else to ensure there are always two replicas up and running. You can check which node each Pod has been allocated on with `kubectl get pod -o wide`. In our case, the minikube cluster has only one node, so both instances will be deployed together.

If you're curious, you can also try to update the application-staging.yml file, apply the Kustomization to the cluster again (`kubectl apply -k .`), and see how the Catalog Service Pods are restarted one after the other (*rolling restarts*) to load the new ConfigMap with zero downtime. To visualize the sequence of events, you can either use Octant or launch this command on a separate Terminal window before applying the Kustomization: `kubectl get pods -l app=catalog-service --watch`.

When you're done testing the application, you can terminate the port-forwarding process with Ctrl-C and delete the cluster with `./destroy-cluster.sh` from polar-deployment/kubernetes/platform/development.

Now that you've learned the fundamentals of configuring and deploying Spring Boot applications in Kustomize, it's time to go to production. That's what the next chapter is all about.

Polar Labs

Feel free to apply what you've learned in this chapter to all the applications in the Polar Bookshop system. You'll need these updated applications in the next chapter, where we'll deploy everything in production.

1. Disable the Spring Cloud Config client.
2. Define a base Kustomization manifest, and update Tilt and the commit stage workflow.
3. Use Kustomize to generate a ConfigMap.
4. Configure a staging overlay.

You can refer to the Chapter14/14-end folder in the code repository accompanying the book to check the final result (**https://github.com/ThomasVitale/cloud-native-spring-in-action**).

Summary

- A configuration server built with Spring Cloud Config Server can be protected with any of the features offered by Spring Security. For example, you can require a client to use HTTP Basic authentication to access the configuration endpoints exposed by the server.
- Configuration data in a Spring Boot application can be reloaded by calling the `/actuator/refresh` endpoint exposed by Spring Boot Actuator.
- To propagate the config refresh operation to other applications in the system, you can use Spring Cloud Bus.
- Spring Cloud Config Server offers a Monitor module that exposes a `/monitor` endpoint that code repository providers can call through a webhook whenever a new change is pushed to the configuration repository. The result is that all the applications affected by the configuration change will be triggered by Spring Cloud Bus to reload the configuration. The whole process happens automatically.
- Managing secrets is a critical task of any software system, and it is dangerous when mistakes are made.
- Spring Cloud Config offers encryption and decryption features for handling secrets safely in the configuration repository, using either symmetric or asymmetric keys.
- You can also use secrets management solutions offered by cloud providers like Azure, AWS, and Google Cloud and leverage the integration with Spring Boot provided by Spring Cloud Azure, Spring Cloud AWS, and Spring Cloud GCP.
- HashiCorp Vault is another option. You can either use it to configure all Spring Boot applications directly through the Spring Vault project or make it a backend for Spring Cloud Config Server.
- When Spring Boot applications are deployed to a Kubernetes cluster, you can also configure them through ConfigMaps (for non-sensitive configuration data) and Secrets (for sensitive configuration data).
- You can use ConfigMaps and Secrets as a source of values for environment variables or mount them as volumes to the container. The latter approach is the preferred one and is supported by Spring Boot natively.
- Secrets are not secret. The data contained within them is not encrypted by default, so you shouldn't put them under version control and include them in your repository.
- The platform team is responsible for protecting secrets, such as by using the Sealed Secrets project to encrypt secrets and make it possible to put them under version control.

- Managing several Kubernetes manifests to deploy an application is not very intuitive. Kustomize provides a convenient way to manage, deploy, configure, and upgrade an application in Kubernetes.
- Among other things, Kustomize provides generators to build ConfigMaps and Secrets, and a way to trigger a rolling restart whenever they are updated.
- The Kustomize approach to configuration customization is based on the concepts of bases and overlays.
- Overlays are built on top of base manifests, and any customization is applied via patches. You saw how to define patches for customizing environment variables, ConfigMaps, container images, and replicas.

Previous chapter

< 13 Observability and monitoring

Next chapter

15 Continuous delivery and GitOps >